

Atelier en opti avancée Projet 1

Antonin Sylvestre
Thomas Blanché

February 2026

Table des matières

1	Introduction	2
2	Partie 1 : modélisation	2
2.1	Explication des méthodes de modélisation pertinentes de fonctions	2
2.1.1	régression polynomiale	3
2.1.2	L'interpolation par splines	3
2.1.3	Une bonne utilisation des données	4
2.2	Modelisation	5
2.2.1	Méthodologie	5
2.2.2	Résultats	8
3	Partie 2 : Programmation Dynamique	10
3.1	la programmation dynamique	10
3.1.1	Principes fondamentaux	10
3.1.2	Fonctionnement	10
3.1.3	Application à nos turbines	11
3.2	Méthodologie	12
3.2.1	notre programmation dynamique	12
3.2.2	implémentation C++	12
3.2.3	Optimisations de notre implémentation C++	14
3.3	Résultats	16
4	Discussion	17
5	Conclusion	18
6	Annexes	19
7	Bibliographie	23

1 Introduction

La production électrique du Canada est en très grande partie basé sur l’hydroélectrique (58 % de la production nationale). Ce secteur est donc critique pour le Canada. L’exploitation d’une centrale hydroélectrique est complexe et demande des calculs d’optimisation pour assurer son bon fonctionnement. La complexité vient notamment des multiples variables qui évoluent autour et à l’intérieur du barrage. Par exemple la “hauteur de chute nette”, qui dépend de la hauteur en amont, de la hauteur en aval et des frottements de l’eau dans le barrage qui dépendent eux même du débit de l’eau... Tout ceci étant influencé par notre gestion du barrage, dont nous souhaitons exploiter les capacités de manière optimale. Les barrages sont composés de plusieurs turbo-alternateurs dont l’efficacité varie. Ainsi, pour chaque environnement et dans chaque situation, la répartition du débit entre les différentes turbines permettant de maximiser la puissance totale générée diffère. sources : [1]

Dans le cadre de ce projet, nous allons développer un outil de gestion pour une centrale hydroélectrique. Son objectif principal est de déterminer quelles turbines mettre en marche ainsi que le débit envoyé à chaque turbine dans le but de maximiser la puissance totale produite, et donc d’optimiser notre production.

Ce rapport s’inscrit dans une suite de 3 projets. Dans ce premier projet, nous irons jusqu’au développement de la partie Programmation dynamique de l’outil. Ce premier projet se divise lui même en deux grande partie :

- Partie 1 : Modélisation des fonctions de production. En partant d’un tableau de valeurs de production réelles (un historique de données entre 2013 et 2017) pour une centrale avec 5 turbines, nous allons modéliser l’élévation avale en fonction du débit total, et la puissance produite par chaque turbine en fonction de la hauteur de chute nette et du débit turbiné. Ainsi nous serons capables de modéliser ladite centrale hydroélectrique.
- Partie 2 : Programmation dynamique. Dans cette deuxième partie l’objectif va être l’implémentation d’un algorithme de programmation dynamique en utilisant les fonctions de production trouvées dans la partie 1. Le tout permettra de répartir un débit cible donné à la centrale hydroélectrique entre chacune de ses turbines afin d’optimiser la production, tout en respectant les contraintes données.

2 Partie 1 : modélisation

2.1 Explication des méthodes de modélisation pertinentes de fonctions

Pour pouvoir réaliser notre programmation dynamique, nous n’allons pas pouvoir utiliser directement les valeurs de l’historique de production de la central données. Les algorithmes de résolution (comme la programmation dynamique) nécessitent des fonctions mathématiques continues ou discrètes, permettant d’obtenir un résultat à partir d’une situation de départ. Par exemple : J’envois tant d’eau dans une turbine, avec telle hauteur de chute, quelle sera

la puissance que je peux en tirer ? Pour créer ces fonctions nous devons transformer les données brutes provenant de capteurs en des fonctions mathématiques s'en rapprochant un maximum, afin de coller à la réalité autant que possible. Cette opération se nomme le *curve fitting*. Dans le cadre de ce projet, nous en aurons besoin de deux : une bidimensionnelle (élévation aval selon le débit total) et une tridimensionnelle (puissance d'une turbine selon la hauteur de chute nette et le débit turbiné). Il y a plusieurs méthodes de modélisation. Voici deux possibilités :

2.1.1 régression polynomiale

La régression polynomiale est une méthode couramment utilisée pour modéliser des données physiques. Cela consiste à ajuster un polynôme de degré n aux points de données. L'objectif de cette méthode est de minimiser le carré de la distance entre chaque point de données expérimentale et le polynôme que l'on ajuste (méthode des moindres carrés totaux). source : [2]

Dans la réalisation de cette méthode, l'étape la plus critique est le choix du degré du polynôme. Avec un degré trop faible, nous aurons un sous-apprentissage : la courbe ne suivra pas assez les variations des données créant alors une forte différence entre notre fonction théorique et les données pratiques, une courbe pourrait par exemple devenir une droite. Dans le cas du choix d'un degré trop élevé, il y a un risque de sur-apprentissage : la fonction générée suivra les données de manière trop fidèle. Au lieu de modéliser la tendance des données qui permet de prédire des valeurs qui n'ont jamais été mesurées, la fonction sera non seulement trop influencée par de petites différences entre des points dû à la marge d'erreur des capteurs, mais risquera en plus de prendre des valeurs trop extrêmes : dans l'idée, des points de données alignés pourraient être modélisés par une sinusoïde qui passerait par chacun de ces points, un peu comme dans la figure 1 . La fonction perd donc tout son intérêt.

source : [3]

Il existe de nombreux outils pour réaliser des régressions polynomiales :

- Python rend cela possible grâce à l'utilisation de plusieurs bibliothèques, telles que Scikit-Learn et NumPy.
- Excel permet aussi la réalisation de régressions polynomiales, quoique son utilisation avec des données en 3D est laborieuse et peu recommandée.
- Très couramment employé dans le domaine des statistiques, le langage de programmation R est particulièrement puissant pour l'analyse statistique. Il est également très complet pour l'exécution de régressions linéaires.
- Enfin, MATLAB offre la possibilité d'effectuer des régressions linéaires facilement, avec une excellente visualisation. Cet outil est disponible directement en ligne, et ne nécessite donc aucune installation préalable.

2.1.2 L'interpolation par splines

Contrairement à la régression polynomiale qui cherche une équation globale pour l'ensemble des données, l'interpolation par splines divise les données en plusieurs intervalles et effectue une sorte de mini régression polynomiale avec des polynômes de faible degré sur chacun de ces intervalles. C'est très utile lorsque l'on travaille sur des données ou l'on peut distinguer

plusieurs “étapes”. Comme par exemple si l’énergie produite par les turbines chutait d’un coup après être passé sous une certaine hauteur de chute alors qu’elle diminuait doucement au-dessus de cette hauteur. On peut alors, grâce à l’interpolation par splines, identifier deux comportements différents et les modéliser chacun de leur côté pour ensuite les fusionner tout en garantissant que la courbe finale soit continue et ce même au point de jonction. Sans cela, avec seulement une régression polynomiale classique, les deux comportement d’une courbe pourrait nous forcer à trop augmenter le degré du polynôme et risquer un sur apprentissage comme mentionné dans la partie précédente.

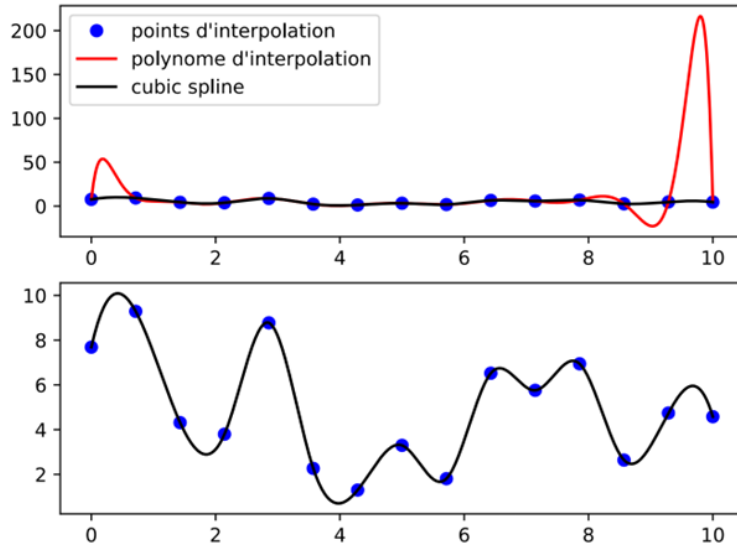


FIGURE 1 – Spline Curve Fitting

source : [4]

Comme on peut voir dans ce graphique (exemple repris du cours Mathématiques Numériques par Bruno Pinçon et J.-F. Scheid p22), l’interpolation par splines (en noir) fonctionne parfaitement malgré les changements de comportement des points d’interpolation alors que le polynôme a lui besoin d’avoir un degré si haut qu’il est finalement sujet à de fortes oscillations.

2.1.3 Une bonne utilisation des données

Quel que soit l’outil ou la méthode choisie, les données ne doivent pas être utilisées telles qu’elles proviennent de capteur expérimental. Il faut réaliser un prétraitement de détection et gestion des valeurs aberrantes. Par exemple des puissances négatives, ou débit négatif pour des turbines.

2.2 Modelisation

2.2.1 Méthodologie

Pour ce projet, nous avons choisi d'utiliser la version en ligne de MATLAB car elle permet d'accéder facilement et rapidement à un environnement parfait pour faire des régressions polynomiales, notamment grâce à l'outil Curve Fitter. MATLAB permet aussi une visualisation interactive des points de données et des fonctions que nous faisons ce qui permet une bien meilleure compréhension et un confort apprécié.

Dans le cadre de ce projet, pour réaliser la programmation dynamique de la partie 2, nous avons besoin de modéliser plusieurs fonctions différentes :

- Premièrement, une fonction donnant l'élévation en aval en fonction du débit total d'eau turbiné. C'est donc une fonction bidimensionnelle.
- Dans un second temps, cinq fonctions donnant la puissance produite en fonction de la hauteur de chute nette (que l'on sait calculer) et du débit turbiné. Cela fait une fonction tridimensionnelle pour chaque turbine.

Pour le calcul du niveau aval en fonction du débit total. Déjà, nous partons du principe que le niveau en aval dépend uniquement du débit passant le barrage. Cela fonctionne pour 2 raisons : quand on fait passer plus d'eau, l'eau se répartit dans un plus grand volume, en prenant plus de hauteur dans la vallée qui a une forme de V, ce qui permet aussi à l'eau de s'écouler plus rapidement, donc de moins influencer sur le niveau de l'eau futur. De plus, en principe, le régime du barrage n'est pas censé changer brusquement en permanence, ce qui permet de lisser le niveau de l'eau, et donc d'avoir une modélisation plus précise dans le mode de fonctionnement par défaut (changements de débits peut fréquents et/ou lent).

Ensuite, nous avons testé plusieurs formules différentes. Après avoir visualisé les données nous nous sommes rapidement orienté vers une racine carré ou un logarithme. Avec une formule sous la forme $p0 * rac(x) + p1$, la courbe ne parvenait pas à suivre les points de données les plus à droite : la hauteur de chute croît moins rapidement que la racine carrée. Nous sommes passés au logarithme : $p0 * log(x) + p1$. Malgré cela, la courbe décrochait toujours des points lorsque l'on dépassait les $2000m^3/s$ de débit total, car il y a peu de données avec des valeurs aussi extrêmes. Nous avons alors augmenté les poids des données dépassant les $2000m^3/s$ et cela nous a permis de mieux coller aux forts débits sans faire baisser la R-value.

Nous avons poussé encore un peu plus loin en donnant à MATLAB la formule $p0 * log(x) + p1 * rac(x) + p2$ où $p1$ prend une valeur négative, ce qui fait descendre la courbe pour les plus hauts debits. La R-value était très légèrement meilleure mais nous avons gardé le logarithme en raison de sa simplicité et de sa R-value, déjà très satisfaisante (cf partie résultat pour voir le graphique de la courbe).

Passons au calcul de la puissance produite en fonction de la hauteur de chute nette et du débit turbiné. Nous avons constaté qu'en isolant uniquement la puissance en fonction du débit, la courbe ressemble à une sinusoïde entre $-\pi/2$ et $\pi/2$, avec une croissance qui est faible pour des débit proche de 0, augmente jusqu'autour $60m^3/s$, et diminue à nouveau.

Ceci est particulièrement visible sur les turbines 3 et 5. Cette forme peut être approximée en soustrayant un cube à un carré avec des facteurs adéquats. Ensuite, en isolant la puissance en fonction de la hauteur de chute nette, on constate que la puissance à débit fixe augmente avec la hauteur de chute, mais les données ne sont pas assez diverses pour reconnaître un motif particulier. Étant donné la complexité du problème, nous avons “simplement” opté pour un polynôme. Après plusieurs expérimentations, entre des polynômes de degré 2 et 4 le polynôme bivarié de degré 3 est ressorti. Il est de la forme suivante :

$$puissance(x, y) =$$

$$p00 + p10 * x + p01 * y + p20 * x^2 + p11 * x * y + p02 * y^2 + p30 * x^3 + p21 * x^2 * y + p12 * x * y^2 + p03 * y^3$$

où x représentant le débit et y la hauteur de chute nette. Nous utilisons ce polynôme, car nous avons constaté qu’il permettait de maximiser la R-value en gardant une extrapolation plausible dans nos bornes, à savoir débit entre 0 et $180 \text{ m}^3/\text{s}$, et hauteur de chute entre 25m et 35m.

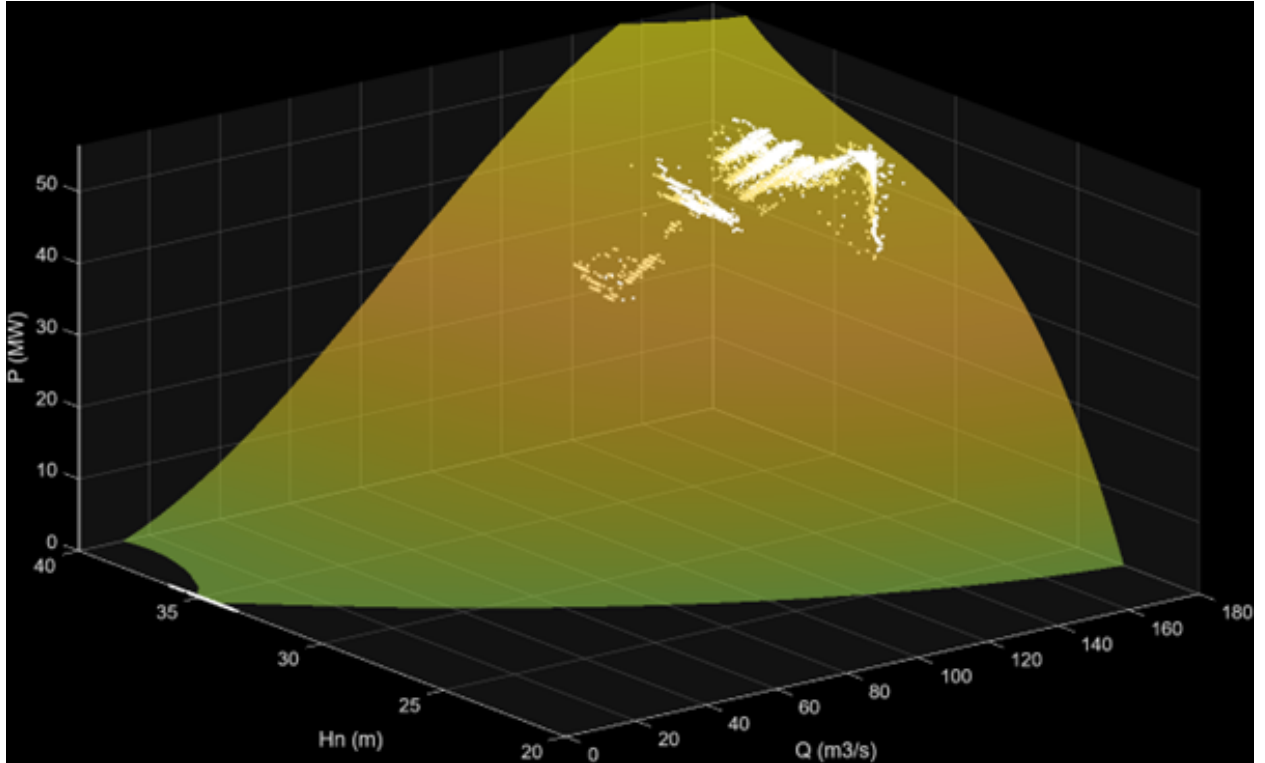


FIGURE 2 – fonction de production de la turbine 1 - avant ancrage

Nous avons également essayé d’autres forme d’équations comme $p0 + p00 * atan(p01 * x + p02) + p03 * x + p10 * log(y - 10) + p11 * y + p111 * log(x + 1) * log(y + 1)$, en ajoutant des contraintes sur les coefficients, mais n’avons pas réussi à générer de fittings vraiment satisfaisant dans notre domaine réalisable, bien que les solutions asymptotiques semblaient plus pertinentes.

En utilisant seulement les données fournies, nous avons constaté que nos fonctions d'approximations étaient imprécises pour des débits faibles (puissance inférieure à 0 sous $50\text{m}^3/\text{s}$ de débit), et qu'elle n'était pas suffisamment stable dans certains cas extrêmes (débit > 160 et Hauteur de chute nette $> 35\text{m}$). Ce problème est visible sur la figure 2. Pour pallier ces 2 problèmes, nous avons augmenté les données réelles avec des "points d'ancrage". Ces points sont placés régulièrement le long de l'axe de la hauteur de chute, pour un débit de 0, et permettent de créer une courbe plus régulière sur les petites puissances... et résout également le problème de l'instabilité.

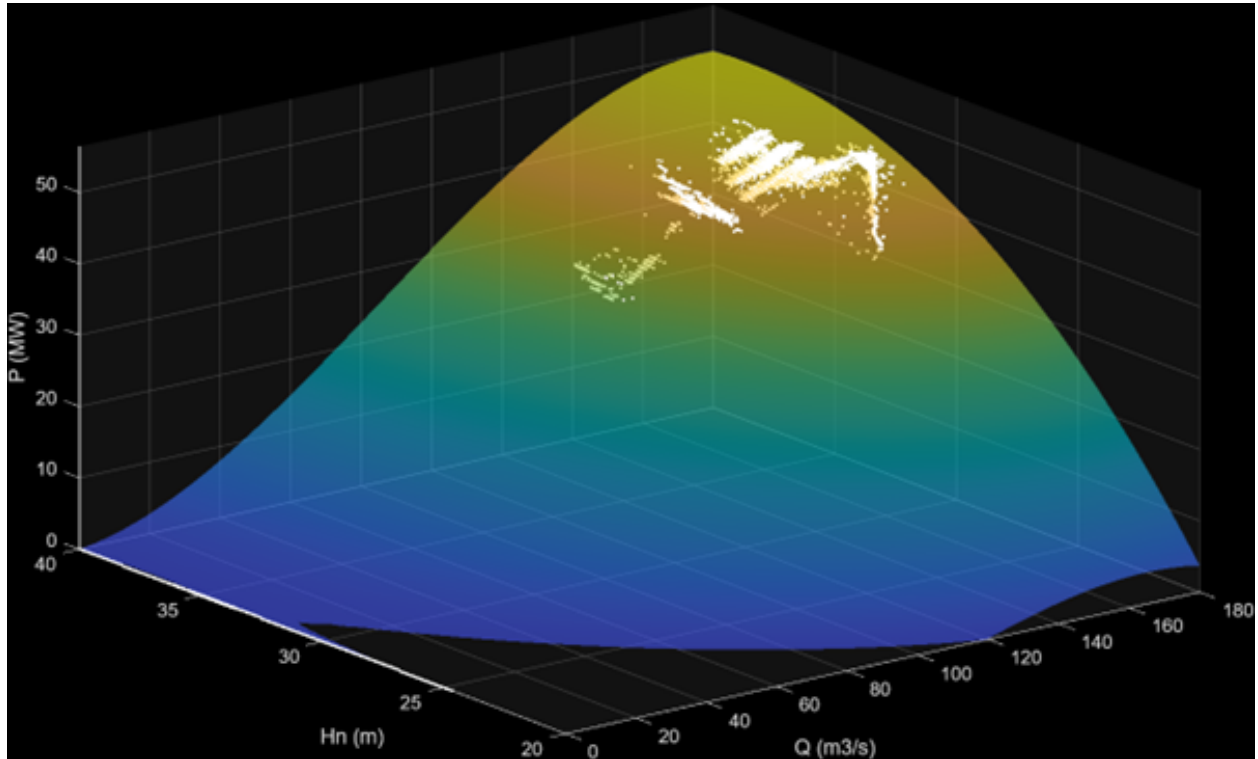


FIGURE 3 – fonction de production de la turbine n°1 - après ancrage

2.2.2 Résultats

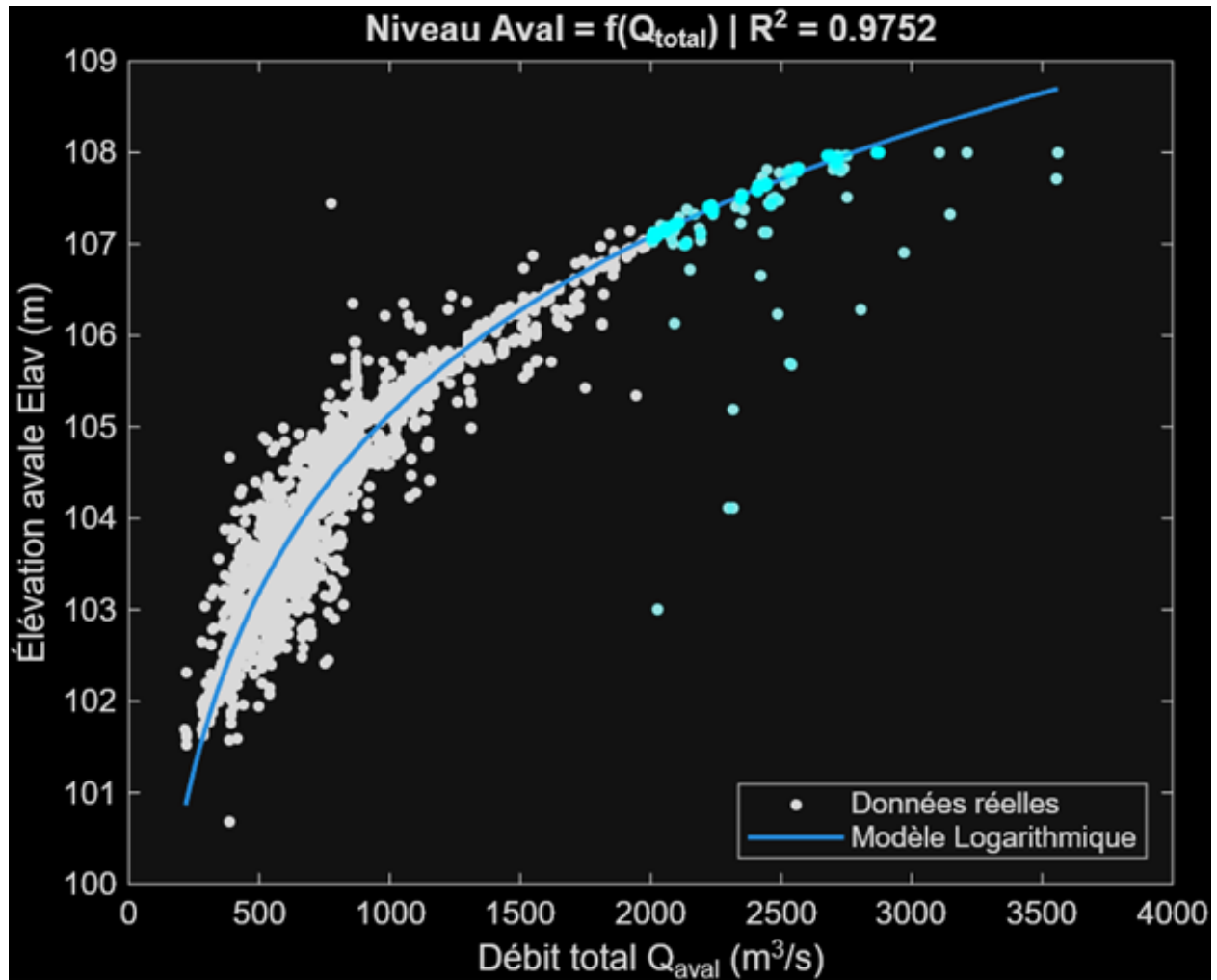


FIGURE 4 – Niveau Aval en fonction du débit

Dans la figure 4 les points bleus sont les points de données dépassant les 2000 de débit dont le poids a été multiplié par 100.

Élévation en aval en fonction du débit total d'eau turbiné voici la formule et les valeurs retenus pour la première courbe :

$p0 * \log(x) + p1$ avec $p0 = 2.805$ et $p1 = 85.76$

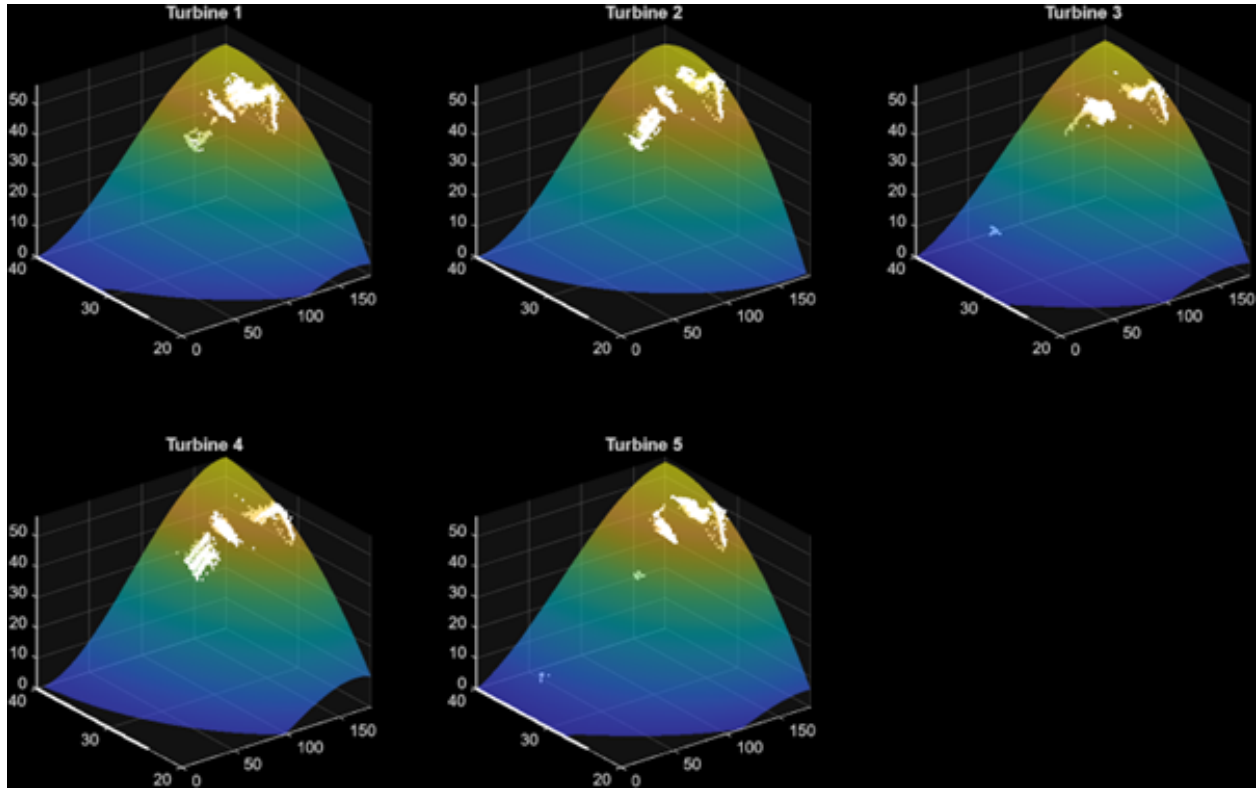


FIGURE 5 – Fonction de production de chaque turbine

TABLEAU 1 – Coefficients du modèle poly33 pour les cinq turbines.

Paramètre	Turbine 1	Turbine 2	Turbine 3	Turbine 4	Turbine 5
R^2 réel	0.9998	0.9998	0.9999	0.9971	0.9998
p_{00}	-2.624	-1.699	-0.758	0.3147	-0.3513
p_{10}	-1.381	-1.954	-1.215	-1.263	-1.366
p_{01}	0.2794	0.1789	0.06992	-0.05313	0.01318
p_{20}	6.02×10^{-3}	7.711×10^{-3}	4.243×10^{-3}	7.074×10^{-3}	5.064×10^{-3}
p_{11}	0.06592	0.09284	0.06423	0.0543	0.06935
p_{02}	-9.15×10^{-3}	-5.728×10^{-3}	-1.757×10^{-3}	2.703×10^{-3}	7.486×10^{-4}
p_{30}	-1.597×10^{-5}	-1.933×10^{-5}	-1.197×10^{-5}	-2.047×10^{-5}	-1.186×10^{-5}
p_{21}	-4.901×10^{-5}	-6.597×10^{-5}	-3.24×10^{-5}	-3.824×10^{-5}	-5.668×10^{-5}
p_{12}	-7.456×10^{-4}	-1.106×10^{-3}	-7.572×10^{-4}	-5.816×10^{-4}	-7.641×10^{-4}
p_{03}	9.436×10^{-5}	5.715×10^{-5}	1.064×10^{-5}	-4.155×10^{-5}	-2.412×10^{-5}

la Figure 5 et le tableau 1 nous permettent d'estimer la cohérence de nos résultats. Le premier constat est que la R-value est très élevée : à priori, nos fonctions de productions sont extrêmement précises. Visuellement, on constate que la surface a une forme acceptable sur chaque turbine, mais qu'elle passe en deçà de 0 quand le débit et la hauteur de chute sont trop faibles. Nous constatons aussi que la décroissance de la courbe est assez élevée : quand la puissance ou la hauteur de chute croît trop (significativement au-dessus de 170), la valeur

de puissance prédite commencera à baisser, ce qui n'est pas réaliste mais hors de bornes de notre problème.

3 Partie 2 : Programmation Dynamique

3.1 la programmation dynamique

La programmation dynamique est une méthode algorithmique et mathématique. Elle est conçue pour résoudre des problèmes d'optimisation en les décomposant en plusieurs sous-problèmes plus simples et interdépendants. Développée autour des années 1950 par le mathématicien Richard Bellman, cette approche est particulièrement adaptée aux situations nécessitant une prise de décision en plusieurs étapes. Contrairement aux algorithmes gloutons qui font le choix optimal à un instant précis sans se soucier de la suite, la programmation dynamique évalue l'impact global de chaque décision et garantit un résultat optimal.

3.1.1 Principes fondamentaux

La résolution d'un problème par la programmation dynamique repose sur deux propriétés essentielles que le problème doit valider.

Premièrement, le principe d'optimalité de Bellman. Ce principe implique que pour qu'une solution globale soit optimale, il faut que cette solution soit composée de toutes les solutions optimales à la suite de ses sous problèmes.

Deuxièmement, le chevauchement des sous-problèmes. L'algorithme doit avoir à résoudre le même sous-problème plusieurs fois. Ce qui fait la force de la programmation dynamique c'est sa capacité à mémoriser les solutions de ces sous-problèmes afin de ne pas avoir à les recalculer. Cela permet de réduire drastiquement la complexité temporelle, transformant souvent un temps d'exécution exponentiel en un temps polynomial. Par exemple, si l'on prend un algorithme qui calcule les valeurs de la suite de fibonacci de façon récursive, à chaque étape le nombre de calculs à réaliser va doubler. Alors qu'en retenant les solutions déjà calculées dans un tableau, on n'en ajoute qu'un seul, ce qui diminue grandement la complexité (on passe d'une complexité cubique à linéaire). source : [5]

3.1.2 Fonctionnement

Pour résoudre un problème en programmation dynamique, il est nécessaire de définir un modèle mathématique basé sur plusieurs éléments standards.

Les étapes (t) : Le problème est divisé en N étapes. À chaque étape, une décision doit être prise. Dans notre contexte d'allocation de ressource, une étape correspond à une turbine à laquelle on attribue un débit d'eau.

Les états (s_t) : L'état représente la situation du problème au début de l'étape t . Il contient toutes les informations nécessaires pour prendre la décision actuelle et les décisions futures.

L'ensemble des états possibles doit être défini, c'est pour cela que dans notre contexte, nous devons discrétiser les débits d'eau pour permettre la résolution du problème.

Les variables de décision (x_t) : Il s'agit du choix effectué à l'étape t pour l'état s_t . L'ensemble des décisions possible dépend de l'état actuel et est soumis à des contraintes physiques ou opérationnelles (comme des bornes minimales et maximales).

La fonction objectif : A chaque étape t et pour chaque état s_t , le choix x_t doit être réalisé en respectant la fonction objectif. Dans notre cas cela va être de maximiser le gain (la puissance électrique) R_t .

La fonction de récursion (Équation de Bellman) : C'est elle qui lie les valeurs optimales de chaque sous problème et permet de recréer la solution optimale globale. Soit $f_t(s_t)$ la valeur optimale obtenue à l'étape t dans l'état s_t et $X(s_t)$ représente l'ensemble des décisions admissibles pour l'état s_t . On écrit alors l'équation de Bellman de tel sorte :

$$f_t(s_t) = \max_{x_t \in X(s_t)} [R_t(s_t, x_t) + f_{t+1}(s_{t+1})]$$

Dans notre cas : $X(s_t) = \{x_t \in \mathbb{R} \mid Q_{min} \leq x_t \leq \min(Q_{max}, s_t)\}$, où Q_{min} est le débit minimum admissible et Q_{max} le débit maximum pour une turbine.

La résolution s'effectue à rebours (Backward pass), on calcul d'abord les valeurs pour la dernière étape N , puis on remonte jusqu'à la première étape pour trouver la solution optimale globale.

3.1.3 Application à nos turbines

La programmation dynamique est parfaite pour le problème de gestion d'une centrale hydroélectrique abordé dans ce projet. L'objectif est de maximiser la puissance totale produite selon un débit total à répartir et une élévation amont donnée, l'algorithme doit déterminer la meilleure façon de répartir le débit d'eau entre les différentes turbines

En discrétisant l'espace des états (le débit disponible), notre algorithme peut simplement évaluer la fonction objectif construite lors de la modélisation des données réelles (Partie I). Aussi, l'approche permet d'ajouter facilement des contraintes opérationnelles supplémentaires (comme l'indisponibilité d'une turbine) en restreignant simplement l'espace des décisions $X(s_t)$.

Nos données d'entrée sont les suivantes : Le débit Total que la centrale doit consommer, Q_{tot} , et le niveau en amont. à partir de ces deux variables, nous pouvons recalculer le niveau en aval avec la formule suivante qui définit le logarithme défini dans la figure 4. La hauteur de chute est donc $H_{reel} = N_{amont} - N_{aval}$. Pour rappel la fonction de production de chaque turbine, modélisée en partie 1 prend en paramètre le débit qui passe par chaque tur-

bine et la hauteur de chute nette. Nous recalculons cette dernière avec la formule suivante : $H_{net} = H_{reel} - 0.5 \times 10^{-5} \times Q_{t_i}^2$ où Q_{t_i} est le débit qui passe dans cette turbine.

3.2 Méthodologie

3.2.1 notre programmation dynamique

Le problème que nous traitons dans ce projet est un problème d'allocation de ressources pour maximiser un gain (les ressources étant l'eau et le gain étant l'énergie produite). Avant de construire notre algorithme de programmation dynamique, nous allons mettre au clair le modèle de notre problème.

Premièrement, chacune des turbines du barrage vont correspondre à une étape. Pour chaque turbine, nous allons résoudre un sous problème et trouver toutes les solutions optimales en fonction des différents états possibles.

Les états sont nos positions de départ possible à chaque étape. C'est l'eau qu'il nous reste encore à turbiner, n'ayant pas été utilisée par les turbines des étapes précédentes. Typiquement, pour chaque étape, les états initiaux vont varier entre 0 et notre débit total.

Il n'y a qu'une variables de décision dans notre problème pour chaque étape : la quantité d'eau que l'on donne à une turbine. C'est le choix que l'on réalise pour chaque état de chaque étape. Pour une étape et un état donné, ce choix doit toujours être optimal.

Afin de déterminer l'option optimale pour chaque étape et chaque état, nous nous référons à la fonction objectif. Dans le contexte de ce problème, l'objectif est de maximiser la production d'énergie. Pour chaque option considérée, nous calculons le gain énergétique généré à l'étape actuelle. À ce gain s'ajoute la valeur maximale obtenue à l'étape précédente pour l'état résultant de cette option. L'état est défini par le débit d'eau restant à allouer après la décision prise.

Finalement, une fois tous les résultats de tous les choix possibles calculés, nous allons pouvoir construire notre solution optimale globale en remontant nos résultats : On part des choix possibles à la première étape, on récupère le meilleur gain. Ensuite, on connaît l'état du système pour l'étape 2, et on cherche le meilleur gain pour cet état, ce qui nous permet d'avoir l'état pour l'étape 3. On poursuit cet algorithme de manière récursive jusqu'à la fin du problème.

3.2.2 implémentation C++

Il existe de nombreux langages de programmations Orienté Objets qui auraient pu convenir dans le cadre de ce projet. On peut citer Julia, Python, Java ou le C++. Toutes ces options sont valides, et permettent d'obtenir des temps d'exécutions acceptables avec plus ou moins d'effort.

L'implémentation a été réalisée en C++. Ce choix est justifié par plusieurs facteurs. Le C++ est largement répandu, bénéficie d'un écosystème très développé et offre d'excellentes performances ainsi qu'une grande souplesse, à la fois dans l'utilisation du code et dans son contexte d'utilisation.

Grâce à la définition de classe, d'interface et de structure, nous avons pu modéliser le barrage de façon propre. Nous avons notamment séparé les différents rôles dans différentes classes :

- Calculator : Une classe abstraite mathématique qui permet de calculer un résultat à partir de 2 paramètres. L'implémentation du calcul du résultat peut varier selon les besoins de l'utilisateur.
- PolyDeg3 : Cette classe héritant de calculator qui implémente un polynôme bivarié de degré 3, dont les poids sont paramétrables.
- Turbine : Classe représentant une turbine. Elle a un système de blocage, pour "l'éteindre", et déclare un débit maximum (et minimum) autorisé. Elle possède une fonction qui prend en entrée un débit et une hauteur de chute pour calculer la hauteur de chute nette, et utilise le Polynome de degré 3 calculé dans la Partie 1 pour en tirer la puissance produite.
- Centrale : Classe qui représente la centrale hydroélectrique. Elle a un tableau de 5 turbines, et utilise un "ResourceAllocationSolver" pour déterminer la répartition optimale du débit entre ses 5 turbines, en fonction d'un niveau amont et d'un débit total demandé.
- ResourceAllocationSolver : Cette classe abstraite représente un solveur de problèmes de répartition de ressources. Les classes qui en héritent doivent permettre de déterminer comment répartir les ressources entre les états pour maximiser le gain. Elle le fait à partir de fonctions de productions (une par état), d'une quantité de ressources totale et de limites sur les quantité de ressources allouables à chaque état.
- DPResourceAllocation : Cette classe représente implémente ResourceAllocationSolver avec de la Programmation dynamique et en discrétisant la quantité de ressource allouable. Le pas est paramétrable.

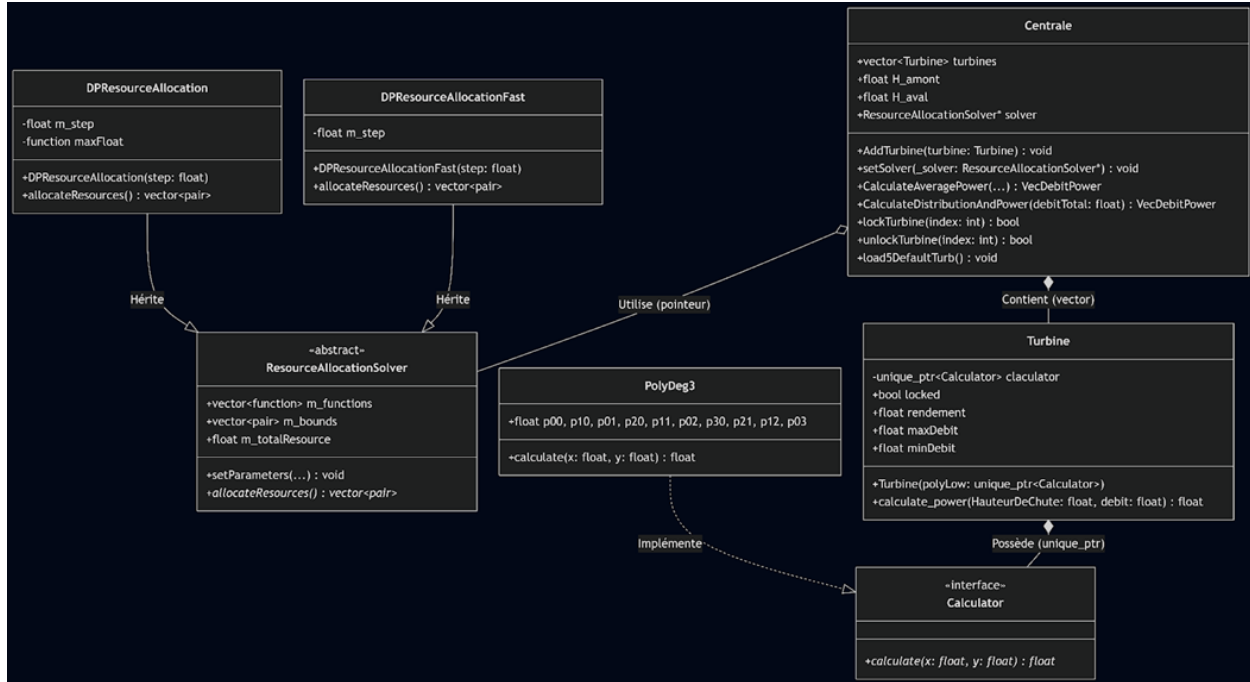


FIGURE 6 – Diagramme de classe de l’implémentation C++

3.2.3 Optimisations de notre implémentation C++

Initialement, notre classe “DPREsourceAllocation” réalisait la programmation dynamique exactement telle que nous l’avons vue en cours : nous créons une structure assez complexe (cf Figure 7) permettant de stocker tous les tableaux que nous remplissons lors d’une résolution manuel. Cette méthode nous a permis d’atteindre l’objectif initial avec un temps de résolution raisonnable.

```

// table de programmation dynamique pour stocker les décisions et les gains
std::vector< //1 : chaque étape
    std::vector< //2 : ligne du tableau de l'étape
        std::pair<float, //ressources restantes (Sn)
            std::vector< // 3 : choix
                std::pair<float, // choix à cette étape
                    float> // 4 : gain total avec les meilleurs choix précédents
            >
        >
    >
> dpTable(m_functions.size());
  
```

FIGURE 7 – Structure utilisée dans la class “DPREsourceAllocation”

Nous avons tout de même fait des modifications sur notre programme avec comme objectif principal de limiter le nombre de calculs en réduisant au maximum la taille des tableaux.

Premièrement, le tableau final n'a pas besoin de calculer plusieurs lignes car aucune ressource n'a été consommée avant cet état, elles sont forcément toutes disponibles. En poursuivant ce raisonnement, on se rend compte que l'avant dernier tableau aussi peut être réduit, car la plage des états initiaux possibles est réduite en positionnant la borne inférieure à la quantité d'eau totale moins le maximum que peut consommer la dernière turbine. On peut ensuite continuer à tronquer le début des tableaux avant même de les remplir. Exemple pour l'avant avant dernier : La borne inférieure des états possible sera positionnée à la quantité totale d'eau moins le maximum de consommation des deux dernière turbine etc ... Il est aussi possible de limiter dans certains cas la taille des tableaux des états en arrêtant le processus de choix de manière proactive lors du dépassement de la limite de ressource allouable à cet état ou de celui des ressources totales disponibles. Cela évite des calculs inutiles et des états impossibles ou non pertinents.

Ces évolutions ont permis un léger gain de vitesse d'exécution. Mais en poussant plus loin nous avons identifié plusieurs points permettant une bien plus grande amélioration des performances.

Premièrement la structure de stockage des tableaux est trop complexe : la gestion mémoire en C++ n'est pas adaptée pour une telle utilisation. Il serait donc préférable de la simplifier au maximum. Cette structure garde le nom des colonne est des lignes des tableaux grâce au "`std::pair<>`", cela implique que pour pouvoir accéder à une ligne en fonction de Sn , nous sommes contraint de parcourir toutes les lignes. Enfin, lors du calcul et de l'ajout d'une nouvelle case dans un tableau, nous devons récupérer le meilleur résultat d'une ligne du tableau précédent. Cette recherche du meilleur résultat nous oblige à parcourir tous les résultats d'une ligne et ceux pour chaque calcul de nouvelle case.

Pour répondre à ces problèmes nous avons modifié la structure de stockage des résultats et la façon d'accéder aux résultats. Ne voulant pas écraser le code déjà fonctionnel nous avons créé une deuxième class implémentant la même interface : "`DPRessourceAllocationFast`". Dans cette dernière, nous n'accédons plus au tableau en le parcourant mais directement en calculant la position de la case rechercher en fonction de la valeur de discrétisation donnée au début du programme. Cela permet de récupérer les valeurs désiré en temps $O(1)$ au lieu de $O(n)$. Cela permet aussi d'alléger la structure stockant les tableaux puisque que l'on retire les deux "`std::pair<>`".

```

struct Cell {
    float bestGain = -1.0f;    // Le gain max pour cette ressource à cette étape
    float choiceMade = 0.0f;  // Le débit choisi pour la turbine actuelle
};

// La matrice : [Nombre de Turbines][Nombre de pas de discrétisation]
const int nbSteps = static_cast<int>(m_totalResource / m_step) + 1;
std::vector<std::vector<Cell>> dpTable(m_functions.size(), std::vector<Cell>(nbSteps));

```

FIGURE 8 – Structure utilisée dans la class “DPRessourceAllocationFast”

La seconde optimisation que nous avons réalisée est que nous ne stockons plus tous les choix possibles à chaque ligne, nous ne gardons directement que le meilleur. Puisque nous devons dans tous les cas parcourir les choix lors de l’écriture, autant n’enregistrer que le maximum. Cela permet notamment de ne pas avoir à parcourir cette ligne lors de la recherche d’un maximum depuis les tableaux suivants. Encore une fois, cette modification allège encore la structure en enlevant une dimension complète. Toute ces modification nous permettent d’arriver à un code est une structure bien plus simple, tel que montré dans la figure 8. Le résultat de cette optimisation est que “DPRessourceAllocationFast” est jusqu’à 70 fois plus rapide sur les mêmes problèmes tout en donnant exactement les mêmes résultats.

Finalement nous avons mis en place du multi threading, en passant par OpenMP. Dans notre code, au sein d’une étape, le calcul des différentes solutions optimales de chaque état est fait en parallèle. Cela permet d’encore multiplier la vitesse d’exécution, et ce par environ 15 dans notre cas.

3.3 Résultats

Lorsque l’on compare les 100 premières lignes du tableau excel (tableaux 3 à 6) avec les résultats de notre machine, la première chose que nous constatons est que le nombre de turbines inactives est toujours le même entre la simulation et la réalité. Ceci est plutôt bon signe : cela signifie que notre algorithme fait fonctionner la centrale à un régime proche de celui déjà mis en place par les opérateurs du barrage.

Ensuite, la production électrique est toujours très proche entre notre centrale simulée et la centrale réelle. Il peut y avoir deux raisons : soit l’opérateur du barrage le fait déjà fonctionner de manière optimale, soit il ne le fait pas et nous sous-estimons la production des turbines.

Notre modélisation a un effet majeur sur la manière d’attribuer les turbines. Observez l’équation de la surface représentant la puissance de la turbine dans le tableau 1. On constate que les termes p_{30} , p_{21} (ainsi que p_{02} et p_{12}) sont négatifs. Par conséquent, quand le débit augmente suffisamment, ils prennent le pas sur les termes de degré inférieur, ce qui fait diminuer la valeur de la fonction (et donc de l’énergie). En mettant une valeur très élevée au débit total, et sans limiter le débit absorbable par une turbine, on constate que les valeurs sélectionnées restent entre 180 et 195 m^3 par seconde. Cela signifie qu’à partir de 180 m^3 par seconde, les résultats de notre simulation ne sont plus fiables. Tout ceci est lié au choix du polynôme

bivarié de degré 3 pour la modélisation de la puissance, comme prévu dans la première partie.

Nous avons également comparé l’impact de la résolution du solveur sur les résultats trouvés :

TABLEAU 2 – Comparaison des effets de la discrétisation

Ligne	Type	Q_1	Q_2	Q_3	Q_4	Q_5	Q_{tot}	P_{tot}	P_1	P_2	P_3	P_4	P_5
4	Reel	158	141	0	140	140	578.0	179.2	45.8	44.2	0	44.8	44.4
	0.2m³/s	0	147.4	138.8	148	143.8	578.0	180.3	0	45.9	41.7	47.2	45.5
	5 m³/s	0	145	140	145	145	575	179.4	0	45.2	42.0	46.3	45.8
	20 m³/s	0	140	140	140	140	560	175.0	0	43.7	42.0	44.8	44.4

Ce qui en ressort, c’est qu’avoir une discrétisation très précise est quasiment inutile : Même si calculer la répartition avec un pas de 0.2m³/s est assez rapide (3.2 secondes), le gain par rapport à 5m³/s (résolution instantané) est minime, avec moins de 1% de différence dans ce cas.

La dernière chose que l’on peut constater, c’est que dans notre programme, quand une turbine est désactivée, c’est toujours la première turbine. La raison est simple : Dans notre simulation, elle est moins efficace que les autres, donc c’est toujours la première à être éliminée.

4 Discussion

Il est important de rappeler que la comparaison avec les données n’est pas aussi pertinente que l’on pourrait le croire : dans ces 100 lignes notre algorithme ne peut pas prendre en compte certaines contraintes qui s’appliquent pourtant bien à la réalité. On peut voir dans les données que parfois certaines turbines sont complètement arrêtées. Cela est dû au débit insuffisant qui implique que la distribution optimale se fait sur seulement 4 turbines. Les résultats théorique que nous obtenons suivent toujours la même logique (nous avons toujours le même nombre de turbines actives) mais la différence se trouve dans le choix de la turbine à l’arrêt. Dans une centrale réelle, ce choix peut dépendre de la quantité d’électricité produite (avec pour objectif de maximiser la production), d’une maintenance sur une turbine ou encore d’une répartition plus équitable entre les turbines (si la turbine X n’a pas fonctionné depuis trop longtemps, on priorise son fonctionnement). Contrairement à cela, dans notre cas théorique, nous choisissons la turbine avec comme unique but la maximisation de la production. C’est donc toujours la même turbine qui est à l’arrêt (turbine 1) car elle est moins efficace que les autres. Cela explique en partie nos résultats en moyenne légèrement plus élevés que la réalité lorsque la discrétisation est a 0.2.

Si les résultats ne sont pas parfaits, c’est aussi dû à l’imperfection des fonctions trouvées en première partie. Le calcul de la production des turbines est une approximations de mesures obtenues par des capteurs pouvant aussi avoir des erreurs. Comme nous calculons le niveau aval uniquement à partir du débit, celui-ci ne tient pas compte du niveau précédent, et manque forcément de précision. Enfin le calcul de la perte de charges est une approximation mathématique simplifiée, qui donne un bon ordre d’idée, sans plus.

Le niveau de discrétisation est sélectionnable pour deux raisons : premièrement, diminuer la discrétisation nous permet d’avoir un résultat plus précis et donc plus proche de ce qui devrait être la solution optimale exacte. Car lorsque le pas est réglé sur 5 par exemple, la répartition sur les turbines n’est faite que par palier de 5. Cela veut aussi dire qu’une petite quantité d’eau ne va pas être distribuée aux turbines ($Q_{tot} \bmod 5$). Dans ce cas très précis ça ne change pas grand chose. Cela dit, pour d’autres problèmes avec de plus grandes variations locales, ça deviendrait plus intéressant.

Pour revenir sur le travail réalisé pour diminuer le temps d’exécution de notre programme, entre la première et la dernière version nous estimons que la vitesse a été multipliée par 1000. Nous sommes passés d’une complexité $O(n^3)$ à $O(n^2)$ voire même $O(n)$ sur une machine avec un nombre de threads non limité, où $n = \text{Pas de discrétisation} * \text{nombre de ressources}$. Ce n’était pas nécessaire, mais il est toujours intéressant de réfléchir au temps d’exécution des programmes. Pour un utilisateur, une réponse instantanée est plus satisfaisante qu’une réponse avec 3 secondes de délai.

5 Conclusion

Dans ce premier projet, nous sommes parvenus à développer ce qui sera la base de notre outil de gestion décisionnelle pour une centrale électrique. C’est-à-dire un programme qui prend en entrée un débit total et le répartit de façon optimale sur les 5 turbines afin de maximiser la production électrique. Pour obtenir ce résultat, nous sommes passés par deux étapes. Premièrement, nous avons modélisé les fonctions de production de chacune des turbines et la fonction donnant l’élévation aval en fonction du débit total, en utilisant l’outil de curve fitting de MATLAB. Ensuite, nous avons modélisé une structure représentant le barrage et l’avons implémentée en C++. Enfin, nous avons mis en place un algorithme de programmation dynamique calculant de façon optimale la répartition de l’eau entre les turbines.

Comme nous avons pu le voir dans la partie résultat, la comparaison avec les 100 lignes de données réelles nous permet de dire que notre simulation est fiable, puisque lorsque des valeurs de débits similaires sont observées, la puissance de la centrale est similaire avec toujours le même nombre de turbines actives.

Ce projet s’inscrit dans une suite de 3 projets : dans le projet suivant nous utiliserons NOMAD pour optimiser la production de notre centrale. Celui-ci étant un programme externe, il nous faudra faire fonctionner notre programme de manière un peu différente. Enfin l’objectif sera de réaliser une application faisant interface avec l’utilisateur permettant le paramétrage de la centrale (minimum et maximum d’eau turbiné pour chaque turbine, hauteur de l’eau en amont, quantité total d’eau à turbiner, le pas de discrétisation) et la visualisation la recommandation optimale calculée par le solveur sélectionnée (Programmation dynamique ou Boîte noire). L’objectif principal de cette interface sera de permettre à n’importe quelle personne (technique ou non) d’utiliser facilement l’outil et d’obtenir une recommandation rapidement.

6 Annexes

TABLEAU 3 – Comparaison des débits (m^3/s) et puissances (MW)

Ligne	Type	Q_1	Q_2	Q_3	Q_4	Q_5	Q_{tot}	P_{tot}	P_1	P_2	P_3	P_4	P_5
4	Reel	158	141	0	140	140	578.0	179.2	45.8	44.2	0	44.8	44.4
	Calcul	0	147	139	148	144	578	180.3	0	45.8	41.8	47.2	45.5
5	Reel	160	141	0	140	140	580.3	179.5	46.2	44.2	0	44.8	44.4
	Calcul	0	148	139	148	145	580	180.7	0	46.1	41.8	47.1	45.8
6	Reel	158	141	0	140	140	578.6	179.1	45.7	44.2	0	44.8	44.4
	Calcul	0	147	139	148	144	578	180.2	0	45.8	41.8	47.2	45.5
7	Reel	160	141	0	140	140	580.2	179.5	46.2	44.2	0	44.8	44.4
	Calcul	0	148	139	148	145	580	180.7	0	46.1	41.8	47.1	45.8
8	Reel	154	141	0	140	140	574.0	178.3	44.8	44.2	0	44.9	44.4
	Calcul	0	147	137	147	143	574	179.2	0	45.8	41.2	46.9	45.3
9	Reel	156	141	0	140	140	576.2	178.7	45.3	44.2	0	44.8	44.4
	Calcul	0	147	138	148	143	576	179.7	0	45.8	41.5	47.2	45.3
10	Reel	159	141	0	140	140	579.9	179.3	45.9	44.2	0	44.8	44.4
	Calcul	0	148	139	148	144	579	180.4	0	46.1	41.7	47.1	45.5
11	Reel	153	141	0	140	140	573.8	178.0	44.6	44.2	0	44.8	44.4
	Calcul	0	147	137	147	142	573	178.9	0	45.8	41.2	46.9	45.0
12	Reel	147	141	0	140	140	567.1	176.7	43.1	44.2	0	44.9	44.4
	Calcul	0	145	135	146	141	567	177.4	0	45.3	40.7	46.7	44.7
13	Reel	147	141	0	140	140	567.6	176.7	43.1	44.2	0	44.9	44.4
	Calcul	0	145	135	146	141	567	177.4	0	45.3	40.7	46.7	44.7
14	Reel	149	141	0	140	140	569.6	177.2	43.6	44.2	0	44.9	44.4
	Calcul	0	146	136	146	141	569	177.9	0	45.6	41.0	46.6	44.7
15	Reel	148	141	0	140	140	568.9	176.9	43.4	44.2	0	44.9	44.4
	Calcul	0	146	135	146	141	568	177.6	0	45.6	40.7	46.6	44.7
16	Reel	151	141	0	140	140	571.0	177.6	44.1	44.2	0	44.9	44.4
	Calcul	0	146	136	147	142	571	178.4	0	45.5	41.0	46.9	45.0
17	Reel	148	141	0	140	140	568.6	176.9	43.4	44.2	0	44.9	44.4
	Calcul	0	146	135	146	141	568	177.6	0	45.6	40.7	46.6	44.7
18	Reel	150	141	0	140	140	570.7	177.4	43.9	44.2	0	44.9	44.4
	Calcul	0	146	136	147	141	570	178.2	0	45.6	41.0	46.9	44.7
19	Reel	150	141	0	140	140	570.7	177.4	43.9	44.2	0	44.9	44.4
	Calcul	0	146	136	147	141	570	178.2	0	45.6	41.0	46.9	44.7
20	Reel	146	141	0	139	140	565.9	176.2	42.9	44.2	0	44.6	44.5
	Calcul	0	145	134	146	140	565	176.8	0	45.3	40.4	46.7	44.5
21	Reel	150	141	0	140	140	570.4	177.4	43.9	44.2	0	44.9	44.4
	Calcul	0	146	136	147	141	570	178.2	0	45.6	41.0	46.9	44.7
22	Reel	0	141	142	141	141	564.1	176.6	0	44.3	42.5	45.1	44.7
	Calcul	0	145	134	145	140	564	176.6	0	45.3	40.4	46.4	44.5
23	Reel	147	141	0	140	140	567.4	176.7	43.1	44.2	0	44.9	44.5
	Calcul	0	145	135	146	141	567	177.4	0	45.3	40.7	46.7	44.7
24	Reel	147	141	0	140	140	567.4	176.7	43.1	44.2	0	44.9	44.5
	Calcul	0	145	135	146	141	567	177.4	0	45.3	40.7	46.7	44.7
25	Reel	0	141	142	141	141	564.4	176.5	0	44.2	42.5	45.1	44.7
	Calcul	0	145	134	145	140	564	176.5	0	45.3	40.4	46.4	44.5
26	Reel	0	140	137	139	140	555.8	174.7	0	44.0	41.4	44.7	44.5
	Calcul	0	143	131	144	137	555	174.1	0	44.8	39.6	46.1	43.6
27	Reel	0	141	139	140	140	559.5	175.7	0	44.3	42.0	44.9	44.5
	Calcul	0	144	132	145	138	559	175.2	0	45.0	39.9	46.4	43.9
28	Reel	0	141	141	141	141	563.7	176.4	0	44.3	42.3	45.1	44.7
	Calcul	0	145	134	145	139	563	176.3	0	45.3	40.4	46.4	44.2

TABLEAU 4 – Comparaison des débits (m^3/s) et puissances (MW)

Ligne	Type	Q_1	Q_2	Q_3	Q_4	Q_5	Q_{tot}	P_{tot}	P_1	P_2	P_3	P_4	P_5
29	Reel	0	141	140	141	140	561.5	176.1	0	44.3	42.2	45.2	44.5
	Calcul	0	144	133	145	139	561	175.8	0	45.0	40.1	46.4	44.2
30	Reel	0	141	139	140	140	559.5	175.8	0	44.3	42.0	45.0	44.5
	Calcul	0	144	132	145	138	559	175.3	0	45.0	39.9	46.4	43.9
31	Reel	0	141	138	139	140	557.2	175.2	0	44.3	41.7	44.7	44.5
	Calcul	0	143	132	144	138	557	174.7	0	44.7	39.9	46.1	43.9
32	Reel	0	140	138	139	140	557.0	175.0	0	44.0	41.7	44.7	44.5
	Calcul	0	143	132	144	137	556	174.4	0	44.8	39.9	46.1	43.7
33	Reel	0	141	138	140	140	558.5	175.5	0	44.3	41.7	45.0	44.5
	Calcul	0	144	132	144	138	558	175.0	0	45.0	39.9	46.1	43.9
34	Reel	0	140	137	139	139	554.3	174.5	0	44.0	41.5	44.7	44.2
	Calcul	0	143	131	144	136	554	173.9	0	44.8	39.6	46.2	43.4
35	Reel	0	141	141	141	141	563.4	176.5	0	44.3	42.4	45.2	44.7
	Calcul	0	145	134	145	139	563	176.4	0	45.3	40.4	46.4	44.2
36	Reel	0	141	141	141	141	563.7	176.5	0	44.3	42.4	45.2	44.7
	Calcul	0	145	134	145	139	563	176.4	0	45.3	40.4	46.4	44.2
37	Reel	0	141	142	141	141	564.3	176.6	0	44.3	42.5	45.1	44.7
	Calcul	0	145	134	145	140	564	176.6	0	45.3	40.4	46.4	44.5
38	Reel	0	141	141	141	141	563.8	176.5	0	44.3	42.4	45.2	44.7
	Calcul	0	145	134	145	139	563	176.3	0	45.3	40.4	46.4	44.2
39	Reel	0	141	140	141	141	562.1	176.4	0	44.3	42.2	45.2	44.7
	Calcul	0	144	134	145	139	562	176.1	0	45.0	40.5	46.4	44.2
40	Reel	0	141	141	141	141	563.6	176.4	0	44.3	42.3	45.1	44.7
	Calcul	0	145	134	145	139	563	176.3	0	45.3	40.4	46.4	44.2
41	Reel	0	141	142	141	141	564.1	176.7	0	44.3	42.5	45.2	44.7
	Calcul	0	145	134	145	140	564	176.7	0	45.3	40.4	46.4	44.5
42	Reel	0	141	141	141	141	563.3	176.5	0	44.3	42.4	45.2	44.7
	Calcul	0	145	134	145	139	563	176.4	0	45.3	40.4	46.4	44.2
43	Reel	0	141	142	141	141	564.3	176.6	0	44.3	42.5	45.1	44.7
	Calcul	0	145	134	145	140	564	176.6	0	45.3	40.4	46.4	44.5
44	Reel	0	141	138	139	140	558.0	175.4	0	44.3	41.8	44.7	44.6
	Calcul	0	143	132	144	138	557	174.8	0	44.8	39.9	46.2	44.0
45	Reel	137	123	135	121	138	653.4	198.1	40.4	37.1	40.2	37.1	43.3
	Calcul	128	138	120	139	128	653	200.0	37.5	42.7	35.6	44.0	40.3
46	Reel	137	118	136	137	138	665.4	202.6	40.4	35.1	40.5	43.3	43.2
	Calcul	130	140	124	140	131	665	203.6	38.1	43.3	36.9	44.3	41.2
47	Reel	137	119	136	137	138	666.4	202.8	40.4	35.5	40.5	43.3	43.2
	Calcul	130	140	124	141	131	666	203.8	38.0	43.2	36.9	44.5	41.1
48	Reel	137	120	136	137	138	667.2	203.2	40.4	35.8	40.5	43.3	43.2
	Calcul	130	140	125	141	131	667	204.2	38.0	43.2	37.2	44.5	41.1
49	Reel	137	118	136	137	138	665.8	202.5	40.4	35.1	40.5	43.3	43.2
	Calcul	130	140	124	140	131	665	203.5	38.1	43.2	36.9	44.2	41.1
50	Reel	137	119	136	137	138	666.3	202.9	40.4	35.5	40.5	43.3	43.2
	Calcul	130	140	124	141	131	666	203.9	38.1	43.2	36.9	44.6	41.1
51	Reel	152	159	0	140	140	590.9	181.6	44.3	48.3	0	44.8	44.3
	Calcul	0	150	143	150	147	590	183.1	0	46.5	42.8	47.6	46.2
52	Reel	147	141	0	140	140	567.3	176.8	43.2	44.3	0	44.9	44.5
	Calcul	0	145	135	146	141	567	177.4	0	45.3	40.7	46.7	44.8
53	Reel	0	141	142	141	141	564.5	176.7	0	44.3	42.5	45.2	44.7
	Calcul	0	145	134	146	139	564	176.6	0	45.3	40.4	46.7	44.2

TABLEAU 5 – Comparaison des débits (m^3/s) et puissances (MW)

Ligne	Type	Q_1	Q_2	Q_3	Q_4	Q_5	Q_{tot}	P_{tot}	P_1	P_2	P_3	P_4	P_5
54	Reel	0	141	141	141	141	563.9	176.5	0	44.3	42.4	45.2	44.7
	Calcul	0	145	134	145	139	563	176.3	0	45.3	40.4	46.4	44.2
55	Reel	0	141	141	141	141	563.5	176.5	0	44.3	42.4	45.2	44.7
	Calcul	0	145	134	145	139	563	176.4	0	45.3	40.4	46.4	44.2
56	Reel	0	141	141	141	141	563.9	176.5	0	44.3	42.4	45.2	44.7
	Calcul	0	145	134	145	139	563	176.3	0	45.3	40.4	46.4	44.2
57	Reel	0	141	140	141	140	561.3	176.2	0	44.3	42.2	45.2	44.5
	Calcul	0	144	133	145	139	561	175.8	0	45.0	40.2	46.4	44.2
58	Reel	0	141	140	141	141	562.1	176.4	0	44.3	42.2	45.2	44.7
	Calcul	0	144	134	145	139	562	176.1	0	45.0	40.4	46.4	44.2
59	Reel	151	141	0	140	140	571.0	177.8	44.2	44.2	0	44.9	44.5
	Calcul	0	146	136	147	142	571	178.5	0	45.6	41.0	46.9	45.0
60	Reel	156	141	0	140	140	576.1	178.8	45.3	44.2	0	44.9	44.4
	Calcul	0	147	138	148	143	576	179.8	0	45.8	41.5	47.2	45.3
61	Reel	150	141	0	140	140	570.7	177.5	43.9	44.2	0	44.9	44.5
	Calcul	0	146	136	147	141	570	178.2	0	45.6	41.0	46.9	44.7
62	Reel	150	141	0	140	140	570.2	177.5	43.9	44.2	0	44.9	44.5
	Calcul	0	146	136	147	141	570	178.2	0	45.6	41.0	46.9	44.7
63	Reel	150	141	0	140	140	570.6	177.5	43.9	44.2	0	44.9	44.5
	Calcul	0	146	136	147	141	570	178.2	0	45.6	41.0	46.9	44.7
64	Reel	137	117	136	137	138	664.6	202.2	40.4	34.7	40.5	43.3	43.2
	Calcul	130	140	124	140	130	664	203.3	38.1	43.3	36.9	44.3	40.8
65	Reel	137	120	136	137	138	667.6	203.2	40.4	35.8	40.5	43.3	43.2
	Calcul	130	140	125	141	131	667	204.1	38.0	43.2	37.2	44.5	41.1
66	Reel	137	120	136	137	138	667.8	203.2	40.4	35.8	40.5	43.3	43.2
	Calcul	130	140	125	141	131	667	204.1	38.0	43.2	37.2	44.5	41.1
67	Reel	137	119	136	137	138	666.8	202.8	40.4	35.5	40.5	43.3	43.2
	Calcul	130	140	124	141	131	666	203.8	38.0	43.2	36.9	44.5	41.1
68	Reel	137	119	136	137	138	666.6	202.9	40.4	35.5	40.5	43.3	43.2
	Calcul	130	140	124	141	131	666	203.8	38.0	43.2	36.9	44.5	41.1
69	Reel	137	124	136	137	138	671.7	204.7	40.4	37.4	40.5	43.3	43.2
	Calcul	131	141	126	141	132	671	205.3	38.3	43.5	37.5	44.5	41.4
70	Reel	137	120	136	137	138	667.1	203.3	40.4	35.9	40.5	43.3	43.2
	Calcul	130	140	125	141	131	667	204.2	38.0	43.2	37.2	44.5	41.1
71	Reel	137	118	136	137	138	665.1	202.5	40.4	35.1	40.5	43.3	43.2
	Calcul	130	140	124	140	131	665	203.6	38.1	43.3	36.9	44.2	41.2
72	Reel	137	118	136	137	138	665.1	202.6	40.4	35.1	40.5	43.3	43.2
	Calcul	130	140	124	140	131	665	203.6	38.1	43.3	36.9	44.2	41.2
73	Reel	137	128	136	138	138	676.4	206.5	40.3	38.9	40.5	43.6	43.2
	Calcul	132	141	127	142	134	676	206.7	38.6	43.5	37.8	44.8	42.0
74	Reel	157	141	0	140	163	600.8	183.9	45.4	44.1	0	44.7	49.7
	Calcul	0	152	146	152	150	600	185.4	0	47.0	43.5	48.1	46.9
75	Reel	148	141	0	140	140	568.0	177.1	43.4	44.3	0	44.9	44.5
	Calcul	0	146	135	146	141	568	177.7	0	45.6	40.7	46.7	44.8
76	Reel	0	141	142	141	141	564.6	176.8	0	44.3	42.6	45.2	44.7
	Calcul	0	145	134	146	139	564	176.7	0	45.3	40.4	46.7	44.2
77	Reel	152	141	0	140	140	572.0	178.1	44.4	44.3	0	44.9	44.5
	Calcul	0	146	137	147	142	572	178.8	0	45.6	41.3	46.9	45.0
78	Reel	151	141	0	140	140	571.8	177.8	44.2	44.3	0	44.9	44.5
	Calcul	0	146	136	147	142	571	178.5	0	45.6	41.0	46.9	45.0

TABLEAU 6 – Comparaison des débits (m^3/s) et puissances (MW)

Ligne	Type	Q_1	Q_2	Q_3	Q_4	Q_5	Q_{tot}	P_{tot}	P_1	P_2	P_3	P_4	P_5
79	Reel	148	141	0	140	140	568.7	177.2	43.4	44.3	0	44.9	44.5
	Calcul	0	146	135	146	141	568	177.7	0	45.6	40.7	46.7	44.8
80	Reel	150	141	0	140	140	570.2	177.6	43.9	44.3	0	44.9	44.5
	Calcul	0	146	136	147	141	570	178.3	0	45.6	41.0	46.9	44.8
81	Reel	0	141	142	141	141	564.8	176.8	0	44.3	42.6	45.2	44.7
	Calcul	0	145	134	146	139	564	176.7	0	45.3	40.4	46.7	44.2
82	Reel	147	141	0	139	140	566.6	176.7	43.2	44.3	0	44.7	44.5
	Calcul	0	145	135	146	140	566	177.2	0	45.3	40.7	46.7	44.5
83	Reel	150	141	0	140	140	570.6	177.6	43.9	44.3	0	44.9	44.5
	Calcul	0	146	136	147	141	570	178.2	0	45.6	41.0	46.9	44.7
84	Reel	148	141	0	140	140	568.1	177.2	43.5	44.3	0	44.9	44.5
	Calcul	0	146	135	146	141	568	177.7	0	45.6	40.7	46.7	44.8
85	Reel	148	141	0	140	140	568.4	177.2	43.5	44.3	0	44.9	44.5
	Calcul	0	146	135	146	141	568	177.7	0	45.6	40.7	46.7	44.8
86	Reel	147	141	0	140	140	567.1	176.9	43.2	44.3	0	44.9	44.5
	Calcul	0	145	135	146	141	567	177.5	0	45.3	40.7	46.7	44.8
87	Reel	149	141	0	140	140	569.4	177.4	43.7	44.3	0	44.9	44.5
	Calcul	0	146	136	146	141	569	178.0	0	45.6	41.0	46.7	44.8
88	Reel	0	141	138	139	140	557.5	175.5	0	44.3	41.8	44.7	44.6
	Calcul	0	143	132	144	138	557	174.8	0	44.8	39.9	46.2	44.0
89	Reel	136	0	137	139	139	550.4	171.2	40.6	0	41.5	44.8	44.3
	Calcul	0	142	130	143	135	550	172.8	0	44.5	39.3	45.9	43.1
90	Reel	0	140	138	139	140	556.0	175.2	0	44.1	41.8	44.7	44.6
	Calcul	0	143	132	144	137	556	174.5	0	44.8	39.9	46.2	43.7
91	Reel	135	0	136	138	139	548.0	170.3	40.3	0	41.2	44.5	44.3
	Calcul	0	142	128	143	134	547	171.9	0	44.5	38.7	45.9	42.8
92	Reel	135	0	133	137	137	541.7	168.2	40.4	0	40.2	44.1	43.5
	Calcul	0	140	126	142	133	541	170.2	0	43.9	38.1	45.6	42.5
93	Reel	135	0	133	137	139	544.0	168.9	40.3	0	40.2	44.1	44.3
	Calcul	0	141	127	142	133	543	170.8	0	44.2	38.4	45.6	42.5
94	Reel	135	0	136	138	139	547.2	170.3	40.3	0	41.2	44.5	44.3
	Calcul	0	142	128	143	134	547	171.9	0	44.5	38.7	45.9	42.8
95	Reel	135	0	135	137	139	545.4	169.6	40.3	0	40.9	44.1	44.3
	Calcul	0	141	128	142	134	545	171.4	0	44.2	38.7	45.6	42.8
96	Reel	136	0	136	139	139	549.6	170.9	40.6	0	41.2	44.8	44.3
	Calcul	0	142	129	143	135	549	172.5	0	44.5	39.0	45.9	43.1
97	Reel	135	0	134	137	139	544.3	169.3	40.3	0	40.5	44.1	44.3
	Calcul	0	141	127	142	134	544	171.1	0	44.2	38.4	45.6	42.8
98	Reel	135	0	135	138	139	546.6	170.0	40.3	0	40.9	44.5	44.3
	Calcul	0	141	128	143	134	546	171.6	0	44.2	38.7	45.9	42.8
99	Reel	135	0	135	137	139	545.5	169.6	40.3	0	40.9	44.1	44.3
	Calcul	0	141	128	142	134	545	171.4	0	44.2	38.7	45.6	42.8
100	Reel	135	0	134	137	139	544.8	169.3	40.3	0	40.5	44.1	44.3
	Calcul	0	141	127	142	134	544	171.0	0	44.2	38.4	45.6	42.8
101	Reel	135	0	132	137	137	540.6	167.8	40.3	0	39.8	44.1	43.5
	Calcul	0	140	126	142	132	540	169.9	0	43.9	38.1	45.6	42.2
102	Reel	135	0	134	137	137	542.2	168.6	40.3	0	40.6	44.1	43.5
	Calcul	0	140	127	142	133	542	170.5	0	43.9	38.4	45.6	42.5
103	Reel	135	131	135	138	0	538.9	166.6	40.4	40.8	40.9	44.5	0
	Calcul	0	140	125	141	132	538	169.3	0	43.9	37.8	45.3	42.2

7 Bibliographie

Références

- [1] Ressources naturelles Canada. À propos de l'électricité. Consulté en février 2026. URL : <https://ressources-naturelles.canada.ca/source-energie/propos-lelectricite>.
- [2] Wikipédia. Ajustement de courbe. Consulté en février 2026. URL : https://fr.wikipedia.org/wiki/Ajustement_de_courbe.
- [3] École Normale Supérieure. Lesson 2 : Linear regression. 2019. URL : https://www.di.ens.fr/appstat/spring-2019/lecture_notes/Lesson2_Linear_Regression.pdf.
- [4] Bruno Pinçon and J.-F. Scheid. *Mathématiques Numériques, Chapitre 3 : Approximation de fonctions*. URL : https://scheid.perso.math.cnrs.fr/Enseignement/MN/CM_MN_ch3B.pdf.
- [5] Wikipédia. Programmation dynamique. Consulté en février 2026. URL : https://fr.wikipedia.org/wiki/Programmation_dynamique.